



Zadanie 2

Quick sort $O(n \log n)$

Heap sort $O(n \log n)$

Osobné číslo: 121834

1 Zadanie

Porovnanie najviac efektívnosti dvoch zvolených algoritmov triedenia. Zhodnotenie, či sa teoretická zložitosť rovná empirickým výsledkom.

Porovnávané algoritmy:

- Quick sort - $O(n \log n)$
- Heap sort - $O(n \log n)$

2 Analýza metód triedenia

Quick sort

Heap sort

3 Zdrojové kódy

Funkcia QuickSort

```
void QuickSort(int data[],int left, int right)
{
    if(left < right)
    {
        int i = left, j = right;
        // volba pivota na prostredny prvok pola
        pivot = data[(left + right)/2 ];
        do
        {
            while (data[i] < pivot) i++;
            while (pivot < data[j]) j--;
            if (i <= j)
            {
                swap(data[i], data[j]);
                i++;
                j--;
            }
        } while (i <= j);
        QuickSort(data, left, j);
        QuickSort(data, i, right);
    }
}
```

Funkcia Kopa

```
void Kopa(int data[], int N, int i)
{
    int naj = i;
    int lavy = 2*i + 1;
    int pravy = 2*i + 2;
```

```
    if(lavy < N && data[lavy] > data[naj])
    {
        naj = lavy;
    }
    if(pravy < N && data[pravy] > data[naj])
    {
        naj = pravy;
    }
    if(naj != i)
    {
        swap(data[i], data[naj]);
        Kopa(data, N, naj);
    }
}
```

Funkcia HeapSort

```
void HeapSort(int data[], int N)
{
    for(int i = N/2 - 1; i >= 0; i--)
    {
        Kopa(data, N, i);
    }
    for(int i = N - 1; i > 0; i--)
    {
        swap(data[0], data[i]);
        Kopa(data, i, 0);
    }
}
```

Funkcia Check

```
bool Check(int data[], int n)
{
    for(int i = 0; i < n - 2; i++)
    {
        if(data[i+1] < data[i])
            return false;
    }
    return true;
}
```

Funkcia main()

```
int main()
{
    int n;
    for(int j = 1; j <= 10; j++)
    {
        n = j * 1000000;
        int *arr_Quick = new int[n];
        int *arr_Heap = new int[n];
    }
}
```

```
srand(time(NULL));
for(int k=0; k < n; k++)
{
    *(arr_Quick+j) = rand();
    *(arr_Heap+j) = *(arr_Quick+j);
}
double priemer_Quick = 0, priemer_Heap = 0;
for(int i = 0; i < 3; i++)
{
    clock_t t;
    t = clock();
    QuickSort(arr_Quick, 0, n-1);
    t = clock() - t;
    double cas_celkom = ((double)t)/CLOCKS_PER_SEC;
    priemer_Quick += cas_celkom;

    bool okQuick = Check(arr_Quick, n);

    cout<<i+1<<". meranie QUICKSORT pre velkost pola "<<n<<": "<<
        cas_celkom<<" s"<<endl;
    if(okQuick)
    {
        cout<<"Zoradenie OK";
    }
    else
    {
        cout<<"ERORR";
    }
    cout<<endl;
    t = clock();
    HeapSort(arr_Heap, n);
    t = clock() - t;
    cas_celkom = ((double)t)/CLOCKS_PER_SEC;
    priemer_Heap += cas_celkom;

    bool okHeap = Check(arr_Heap, n);

    cout<<i+1<<". meranie HEAPSORT pre velkost pola "<<n<<": "<<
        cas_celkom<<" s"<<endl;
    if(okHeap)
    {
        cout<<"Zoradenie OK";
    }
    else
    {
        cout<<"ERORR";
    }
    cout<<endl;
}
cout<<"Priemerny cas QUICKSORT: "<<priemer_Quick/3<<endl;
cout<<"Priemerny cas HEAPSORT: "<<priemer_Heap/3<<endl;
cout<<endl;
delete [] arr_Quick;
delete [] arr_Heap;
}

return 0;
}
```

4 Overenie triediacich algoritmov

Tabuľky časov zoraďovacích algoritmov

1. algoritmus Quick sort

n [mil]	1	2	3	4	5	6	7	8	9	10
séria										
Mer. 1 [s]	0,091	0,184	0,287	0,385	0,503	0,638	0,874	1,018	1,116	1,346
Mer. 2 [s]	0,088	0,18	0,285	0,385	0,498	0,61	0,917	0,994	1,299	1,286
Mer. 3 [s]	0,087	0,183	0,289	0,38	0,502	0,608	0,996	0,949	1,131	1,423
Čas behu [s]	0,089	0,182	0,287	0,383	0,501	0,619	0,929	0,987	1,182	1,352

2. algoritmus Heap sort

n [mil]	1	2	3	4	5	6	7	8	9	10
séria										
Mer. 1 [s]	0,017	0,033	0,051	0,067	0,084	0,11	0,159	0,163	0,214	0,216
Mer. 2 [s]	0,016	0,03	0,047	0,062	0,078	0,092	0,14	0,153	0,175	0,195
Mer. 3 [s]	0,015	0,031	0,046	0,062	0,078	0,094	0,148	0,153	0,164	0,194
Čas behu [s]	0,016	0,031	0,048	0,064	0,08	0,099	0,149	0,157	0,184	0,202

5 Zhodnotenie triediacich